

Tel Aviv University

Faculty of Engineering, School of Electrical Engineering
Final Engineering Project Report, Project No. 3221



Optical Fibers for Smart Cities

IoT-Enabled Fiber-Optic Sensing for Real-Time Vehicle Tracking and Road-Anomaly Detection

Students

Noa Sapir ID: 208744086

Nitai Dobrecki ID: 315110981

Supervisor

Khen Cohen

Project Carried Out at

School of Electrical Engineering, Tel Aviv University

Academic Year 2025–2026

Table of Contents

Abstract.....	5
1 Introduction.....	6
1.1 Goals of the project.....	6
1.2 Motivation.....	6
1.3 Approach.....	6
1.4 Comparison with existing work.....	7
2 Theoretical Background.....	8
2.1 Distributed Acoustic Sensing.....	8
2.2 Vehicle-induced ground response.....	8
2.3 Multi-sensor fusion and the Kalman filter.....	8
2.4 Anomaly detection without labels.....	9
2.5 Alternative algorithms considered.....	9
3 Simulation and System Design.....	10
3.1 Simulation environment and architecture.....	10
3.2 Vehicle motion model.....	11
3.3 Sensor models.....	11
3.4 Scenario generation.....	12
3.5 Simulation outputs and workflow.....	12
3.6 Assumptions and limitations.....	13
4 Implementation.....	14
4.1 Software architecture.....	14
4.2 End-to-end information flow.....	14
4.3 Communication and dependencies between modules.....	15
4.4 Ground-truth separation and data integrity.....	15
4.5 Key algorithms and engineering decisions.....	16
4.6 The anomaly model in detail.....	16
4.7 Repository structure.....	18
5 Analysis of Results.....	19
5.1 Tracking and fusion accuracy.....	19
5.2 Sensor contribution and fusion behavior.....	21
5.3 Anomaly-detection performance.....	21
5.4 Missed anomalies and failure analysis.....	24
5.5 Comparison with quantitative targets.....	24

5.6 Real-world validation: the Klausner Street experiment.....	24
6 Conclusions and Further Work.....	27
6.1 Results against the original goals.....	27
6.2 Suggestions to improve performance.....	27
6.3 Future development and research.....	27
7 Project Documentation.....	28
7.1 Software.....	28
7.2 Simulations and datasets.....	28
7.3 Documentation contents and location.....	28
8 References.....	29
Appendix A: Feature Catalog.....	30

List of Figures

Figure 1: System block diagram, from the simulated world through multi-sensor sensing, the event bus, tracking and Kalman fusion, to anomaly detection and outputs.....	5
Figure 2: The modeled urban scene. A buried DAS fiber runs beside each road segment, cameras observe intersections, and GPS provides absolute fixes, and the three modalities are fused into a single Kalman track as a vehicle crosses the junction.....	10
Figure 3: Real urban road network imported from OpenStreetMap (North Tel Aviv). Vehicles are simulated along true street geometry, lane counts, and speed limits.....	11
Figure 4: The northern Tel Aviv network divided into 144 simulation regions (tiles). Each region seeds an independent traffic scene, and batch runs across them produce the varied corpus of normal behavior from which the anomaly baseline is learned.....	12
Figure 5: End-to-end information flow: generation, sensing, publication, tracking, fusion, export, feature extraction, and detection.....	14
Figure 6: Anomaly-detection model architecture: feature extraction feeds the trajectory-level Z-score scorer, which produces the per-vehicle anomaly score and flag, while a supplementary CUSUM diagnostic provides temporal localization.....	17
Figure 7: Fused multi-sensor track versus ground truth for a cross-segment vehicle. Top: spatial trajectory with DAS, camera, and GPS measurements, and the dashed line marks the segment boundary. Bottom: instantaneous and rolling 2-D error (RMSE = 0.49 m, max 1.52 m).....	20
Figure 8: Continuous tracking across four connected segments (over-speeding scenario). Top: spatial trajectory with DAS, camera, and GPS measurements and two segment transitions. Bottom: 2-D error over time (RMSE = 0.37 m, max 1.11 m).....	21
Figure 9: Anomaly score of the injected vehicle per scenario, against the $\approx 15.4 \sigma$ decision threshold. Longitudinal and sensor-level anomalies separate clearly from the normal baseline, while subtle lateral and spacing anomalies do not.....	22
Figure 10: Per-vehicle Z-score ranking for the over-speeding scenario. The injected vehicle (red) exceeds the decision threshold by a wide margin, while background vehicles (green) remain in the normal band.....	23

Figure 11: Top anomalous features for the injected vehicle versus the normal training distribution. Dashed lines mark the $\pm 4 \sigma$ envelope, and the vehicle's values (red) lie outside it on speed and lateral-velocity features, explaining the flag.....	23
Figure 12: Location of the Klausner Street experiment beside the Tel Aviv University campus. The red pin marks the camera + DAS measurement point on the real street network imported from OpenStreetMap.	25

List of Tables

Table 1: Sensor measurement models. Each sensor exposes a physically grounded uncertainty that the Kalman filter consumes directly.....	11
Table 2: Feature groups produced by the feature extractor (166 features per vehicle in total).....	17
Table 3: Anomaly-detection results across nine injected scenarios. Detection threshold $\approx 15.4 \sigma$. The injected vehicle was correctly flagged in six of nine cases.....	22
Table 4: Quantitative performance targets and achieved results.....	24
Table 5: Klausner Street experiment, fusing the real DAS recording with a co-located camera. Per-sensor errors are measured against ground truth.....	26

List of Equations

- (1) DAS strain-rate measurement
- (2) Flamant–Boussinesq strain amplitude
- (3) Kalman state vector
- (4) Kalman prediction step
- (5) Kalman update step
- (6) DAS detection probability
- (7) SNR-driven dynamic measurement noise
- (8) Normalized Innovation Squared (NIS)
- (9) Page's CUSUM recurrence
- (10) Per-feature Z-score anomaly criterion
- (11) Position RMSE

Abstract

Modern cities are laced with thousands of kilometers of telecommunication optical fiber. This project addresses a central research question: whether that existing fiber can be used as a distributed urban traffic sensor, and whether the resulting signals can be fused with conventional cameras and GPS to track vehicles and identify irregular driving behavior using only existing fiber infrastructure and complementary sensing modalities. To investigate this, we designed and built **SimStudio**, a complete desktop simulation, sensor-fusion, and anomaly-detection environment for fiber-optic Distributed Acoustic Sensing (DAS) [1], [2].

SimStudio imports real urban road networks from OpenStreetMap [8], drives physically grounded vehicles along them, and synthesizes three complementary sensing modalities: DAS strain on a buried fiber, vision-based camera detections, and satellite GPS fixes. An event-driven pipeline associates these heterogeneous measurements into stable vehicle tracks, fuses them through a six-state constant-acceleration Kalman filter [5], [7] with physically grounded, signal-to-noise-dependent measurement noise, and exports trajectories, error metrics, and diagnostics. A read-only analytics layer then extracts 166 behavioral features per vehicle and detects anomalies using an interpretable per-feature Z-score baseline, with a Page CUSUM signal available as a supplementary temporal diagnostic [6].

The system met its two primary objectives. Fused tracking achieved sub-meter accuracy: a representative cross-segment track reached **0.49 m RMSE** (mean 0.35 m across all evaluation scenarios), well within the project tolerance, and the anomaly model correctly identified the **injected anomalous vehicle in six of nine scenarios**, satisfying the work-plan target of detecting at least five of ten anomalies. The chapters below present the theory, system design, implementation, and results in full, and discuss how the approach can be extended with labeled data and richer DAS physics. As a first step toward physical validation, the system was also exercised on a real DAS + camera recording taken on Klausner Street beside the Tel Aviv University campus, where a recorded bus-stop event was tracked end-to-end by fusing the real DAS and camera measurements through the unchanged pipeline.

SimStudio — Fiber-Optic Multi-Sensor Traffic Monitoring

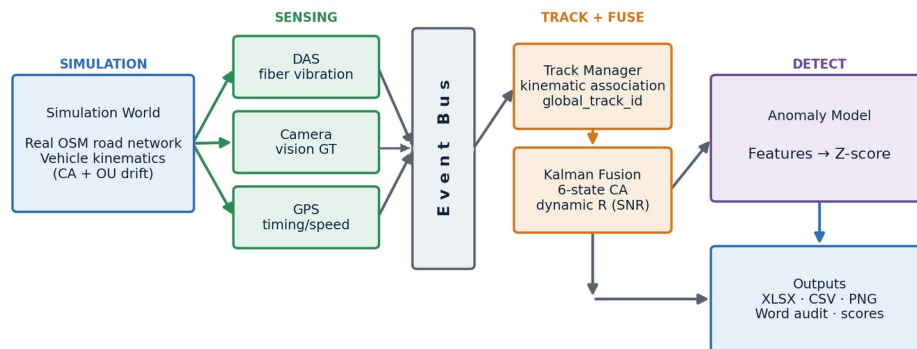


Figure 1: System block diagram, from the simulated world through multi-sensor sensing, the event bus, tracking and Kalman fusion, to anomaly detection and outputs.

1 Introduction

Urban mobility is under increasing strain. Authorities need continuous, low-cost knowledge of how vehicles move through a city and when something goes wrong, such as a sudden stop, a stalled car, a collision, or a wrong-way driver. Conventional monitoring relies on dense networks of cameras and inductive loops, which are expensive to install, costly to maintain, and degraded by adverse weather and darkness. This project pursues a complementary path: re-using the optical-fiber infrastructure already deployed beneath most streets.

1.1 Goals of the project

The project set out to build and validate a multi-sensor traffic-monitoring system with two primary objectives. **First, reliable vehicle tracking** across a monitored road network: maintaining a consistent identity and an accurate trajectory for each vehicle as it travels across multiple street segments, by fusing DAS, camera, and GPS measurements. **Second, accurate anomaly detection**: automatically identifying irregular and unsafe driving behavior, such as speeding, sudden braking, stalled vehicles, collisions, weaving, and tailgating, from the fused sensor signals.

Because dedicated DAS interrogator hardware and an instrumented street were not available within the project timeframe, both objectives were pursued through a physically grounded simulator that reproduces the measurement chain, from vehicle weight to fiber strain to a Kalman measurement, so that algorithms developed against it can transfer to real deployments with limited modification. As a partial bridge to reality, one real DAS + camera recording from Klausner Street was reconstructed and processed through the same pipeline. This experiment is reported in Section 5.6.

1.2 Motivation

Fiber-optic sensing offers properties that complement camera networks, particularly continuous spatial coverage over tens of kilometers from a single cable, robustness to lighting and weather, and low incremental hardware cost because the fiber already exists [1], [3]. Re-using a passive communication asset as a live urban sensor is attractive both scientifically, since it advances multi-sensor spatiotemporal signal fusion, and economically, as a scalable route to city-wide situational awareness and faster emergency response.

1.3 Approach

Rather than treating any single sensor as authoritative, the system is built around complementary fusion. DAS provides dense, weather-independent position and along-fiber speed. Cameras provide accurate but occlusion- and lighting-limited detections. GPS provides absolute position and timing but at low rate and accuracy. A Kalman filter weights each measurement by its physically derived uncertainty, so the estimate relies on whichever sensor is currently most trustworthy [5]. On top of the fused tracks, anomaly detection is framed as learning the distribution of normal driving across many scenarios and then measuring deviation from that norm, an approach that does not require the large labeled anomaly datasets that are scarce in this domain.

1.4 Comparison with existing work

Commercial DAS traffic systems such as OptaSense detect and classify vehicles on single road segments but lack multi-sensor integration and cross-street tracking. Camera-based platforms such as Miovision manage flow at intersections but remain constrained by weather and lighting. Academic work at Tel Aviv University has shown that DAS traffic signals can be learned from co-located video labels [1], [2], and a related study fuses DAS with computer-vision tracking through a Kalman filter [3]. This project extends that line of work by combining all three modalities (DAS, vision, and GPS) into a single cross-segment tracking and anomaly-detection framework, and by packaging it as a reproducible simulation environment that exposes every stage of the pipeline for inspection.

2 Theoretical Background

2.1 Distributed Acoustic Sensing

Distributed Acoustic Sensing transforms a standard telecommunication fiber into a dense array of vibration sensors [1], [2]. A laser pulse is launched into the fiber, and a small fraction is continuously back-scattered (Rayleigh scattering) by intrinsic density variations in the glass. Ground vibration stretches and compresses short fiber segments, shifting the optical phase of the back-scattered light. By comparing the phase between two points separated by a gauge length L , the interrogator recovers the local strain rate (Equation 1):

$$\varepsilon'(t, z) \approx \frac{1}{L} [\dot{u}(t, z+L) - \dot{u}(t, z)] \quad (1)$$

where $u(t, z)$ is the ground displacement along the fiber. Each meter of fiber thus becomes an independent vibration channel, and a passing vehicle produces a characteristic moving signature whose amplitude encodes its weight and whose slope across channels encodes its speed.

2.2 Vehicle-induced ground response

A vehicle is a moving surface load. To first order, the quasi-static strain it induces at a buried fiber at perpendicular distance r follows a Flamant–Boussinesq-type decay (Equation 2):

$$A = \frac{W}{(r + d_0)^2} \quad (2)$$

where W is the vehicle weight and d_0 a small floor that keeps the amplitude finite directly above the cable. This relationship is doubly useful: it lets the simulator generate realistic DAS amplitudes, and it lets the analytics layer back-estimate vehicle weight from the measured amplitude, a feature later used for anomaly detection.

2.3 Multi-sensor fusion and the Kalman filter

The Kalman filter is a standard tool for recursive state estimation under linear-Gaussian assumptions [5], [7]. It maintains a Gaussian belief over a vehicle's kinematic state and alternates a **predict** step that propagates the state through a motion model and an **update** step that corrects it with each incoming measurement, weighted by the relative uncertainty of model and measurement. Because the update weight is the Kalman gain, which is large when the measurement is trustworthy and small when it is noisy, the filter naturally implements complementary fusion: a precise DAS or camera fix corrects the estimate strongly, while a noisy GPS fix only nudges it.

The project uses a six-state constant-acceleration model with decoupled x and y axes. The state vector (Equation 3) encodes position, velocity, and acceleration on each axis:

$$\mathbf{x} = [x \quad v_x \quad a_x \quad y \quad v_y \quad a_y]^T \quad (3)$$

The predict step propagates the state estimate and its covariance through the constant-acceleration transition matrix F and the process-noise covariance Q (Equation 4):

$$\hat{x}_k^- = F \hat{x}_{k-1}, \quad P_k^- = F P_{k-1} F^T + Q \quad (4)$$

When a measurement z arrives, the update step computes the Kalman gain K and corrects the state, with H the observation matrix and R the measurement-noise covariance (Equation 5):

$$K = P_k^- H^T (H P_k^- H^T + R)^{-1} \quad (5)$$

2.4 Anomaly detection without labels

Labeled real-world anomaly data, such as actual collisions, stalls, and reckless maneuvers recorded on DAS, is scarce. Supervised classifiers such as Random Forests or SVMs require many examples of both normal and anomalous behavior and are therefore impractical at this stage. The project instead adopts a one-class, unsupervised approach: learn the distribution of normal driving from many normal scenarios, then score each new vehicle by how far it deviates. A supplementary Page CUSUM diagnostic [6] can monitor the filter's Normalized Innovation Squared over time to indicate *when* a vehicle began behaving abnormally, while the per-feature Z-score baseline scores the whole trajectory to indicate *whether* a vehicle is anomalous overall. As labeled anomaly exports become available, the same feature pipeline could feed a supervised classifier in future work.

2.5 Alternative algorithms considered

Several alternative algorithms were evaluated for the fusion and detection stages before the final design was selected. **Extended and Unscented Kalman filters** handle nonlinear motion and measurement models, but the simulator's kinematics and the position/speed measurements are represented adequately by a linear constant-acceleration model, so the added complexity was not justified at this stage. **Particle filters** are more flexible for strongly nonlinear, non-Gaussian systems, but are computationally heavier and unnecessary given the near-Gaussian measurement noise produced by the sensor models.

Supervised classifiers such as Random Forest or SVM could improve anomaly recall, but they require a larger labeled anomaly dataset than was available, so they are retained as a future option rather than the primary detector. **Deep-learning models** are powerful for interpreting raw DAS waterfalls [2], but demand extensive labeled real data and offer less interpretability than the feature-based approach used here. **Rule-based thresholds** are simple and interpretable, but generalize poorly across traffic densities and sensor layouts. The chosen combination of a linear Kalman filter for fusion and an interpretable Z-score baseline for detection was selected as the best balance of accuracy, interpretability, and data efficiency for the current project scope.

3 Simulation and System Design

This chapter describes the overall system design and the simulation environment that realizes it: the architecture and its rationale, the vehicle and sensor models, scenario generation, the simulation outputs and workflow, and the assumptions and limitations that bound the results.

3.1 Simulation environment and architecture

The system is organized as a one-directional pipeline of loosely coupled stages (simulate, sense, publish, track, fuse, export, analyze) connected by a shared **event bus** (Figure 1). The central design decision was this event-driven decoupling: the simulator publishes typed events (vehicle states and per-sensor measurements) onto the bus, and every downstream component consumes them independently. This approach was preferred over a monolithic design because it makes each sensor and consumer independently testable, lets the tracking, fusion, and analytics layers evolve without modifying the simulator core, and guarantees that the anomaly model remains strictly read-only.

Since dedicated DAS interrogator hardware and an instrumented road segment were not available within the project timeframe, the project focused on a physically grounded simulation environment. The simulator does not attempt to reproduce every optical detail of a DAS interrogator. Instead it reproduces the observable quantities required by the tracking and anomaly-detection algorithms: sensor coverage, measurement uncertainty, detection gaps, DAS amplitude, and SNR-dependent noise. This makes the simulator suitable for controlled algorithm development and evaluation, and means the same feature-extraction and detection code can later be applied to recorded DAS data without modification.

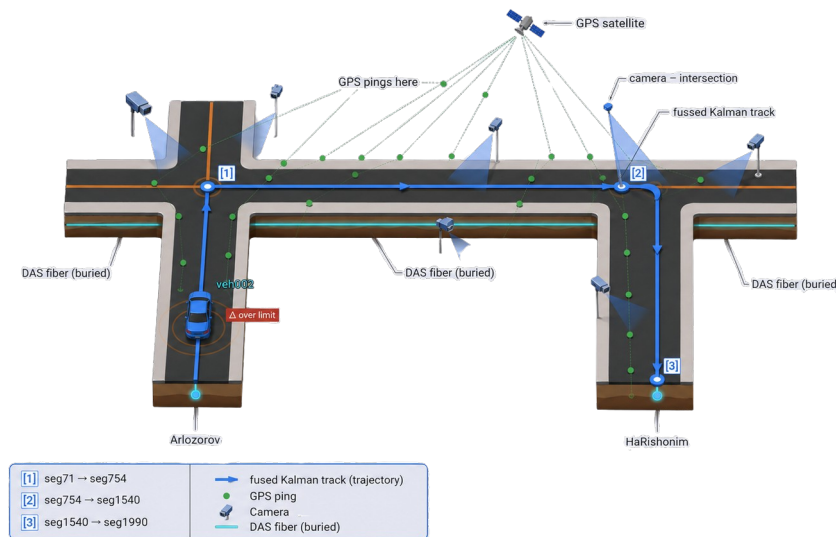


Figure 2: The modeled urban scene. A buried DAS fiber runs beside each road segment, cameras observe intersections, and GPS provides absolute fixes, and the three modalities are fused into a single Kalman track as a vehicle crosses the junction.

The simulation engine advances a world of vehicles on a road network at 25 Hz. The road network is not synthetic: maps are imported from OpenStreetMap [8], so vehicles travel along real streets, for example the Florentin and northern Tel Aviv networks used in evaluation (Figure 3), with true geometry, lane counts, and posted speed limits.

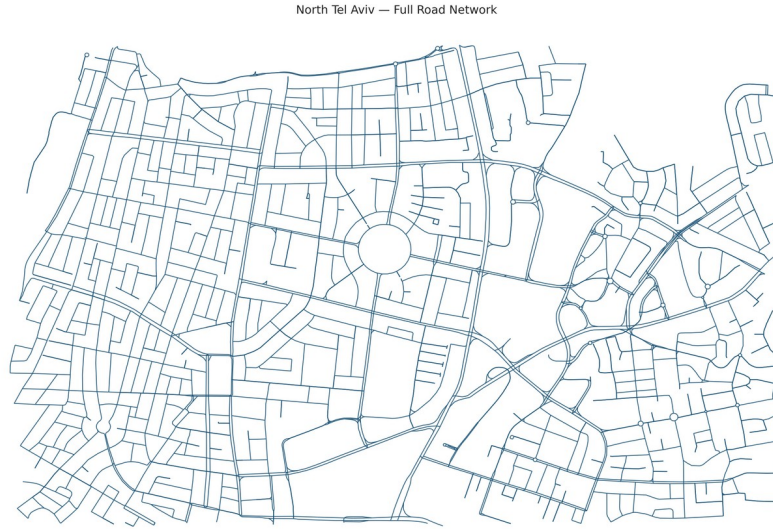


Figure 3: Real urban road network imported from OpenStreetMap (North Tel Aviv). Vehicles are simulated along true street geometry, lane counts, and speed limits.

3.2 Vehicle motion model

Each vehicle is modeled as a point mass on a lane centerline. Its longitudinal motion is driven by a first-order speed-tracking controller toward a stochastic macro speed profile resampled at intervals, producing realistic acceleration and cruise phases. Lateral position follows a mean-reverting Ornstein–Uhlenbeck process bounded to half a lane width, which reproduces natural lane-keeping wander. These ingredients were chosen to reproduce the slightly irregular motion that makes tracking and anomaly detection non-trivial, while remaining inexpensive enough to run hundreds of scenarios.

3.3 Sensor models

After each vehicle moves, the engine generates synthetic measurements from three sensor types (Table 1). The defining feature of all three is that their uncertainty is physically derived and forwarded to the Kalman filter, so fusion weighting is automatic.

Sensor	What it measures	Noise model (key idea)	Rate
DAS	Along-fiber position & speed	$\sigma \propto k/\sqrt{\text{SNR}}$. SNR from weight & distance. Detection gating, bias, peak-merging	~25 Hz
Camera	World position (x, y)	Pinhole projection. Floor + range + off-axis terms. Stochastic FOV gating	~ frame rate
GPS	Absolute position (x, y)	σ grows with range from sensor center. Range-dependent confidence	~1 Hz

Table 1: Sensor measurement models. Each sensor exposes a physically grounded uncertainty that the Kalman filter consumes directly.

DAS is modeled most richly because it is the project's novel modality. Vehicle weight sets the strain amplitude through the Flamant–Boussinesq law. The signal-to-noise ratio then governs both the probability that the vehicle is detected at all and the position uncertainty. Detection follows a sigmoid in SNR^2 , so a vehicle at the threshold SNR is detected half of the time (Equation 6):

$$p_{detect} = \frac{SNR^2}{SNR^2 + t^2} \quad (6)$$

The model also injects a small systematic peak-finding bias and merges the strain peaks of vehicles that are close along the fiber, two effects that mimic well-documented DAS artifacts and keep the simulated data faithful to the technology's limitations.

3.4 Scenario generation

To build a broad notion of normal driving, the northern Tel Aviv network was divided into 144 simulation regions (Figure 4). Each region seeds an independent traffic scene, and batch runs across the regions produce a large, varied corpus of normal behavior. The normal set deliberately spans twelve conditions (sparse to heavy traffic, heavy vehicles, stop-and-go, legal high speed, and sensor layouts from full coverage down to DAS-only) so that the anomaly baseline does not over-fit a narrow definition of normal. A separate set of nine scenarios each injects one anomalous vehicle drawn from eight anomaly categories, providing controlled positives for evaluation.

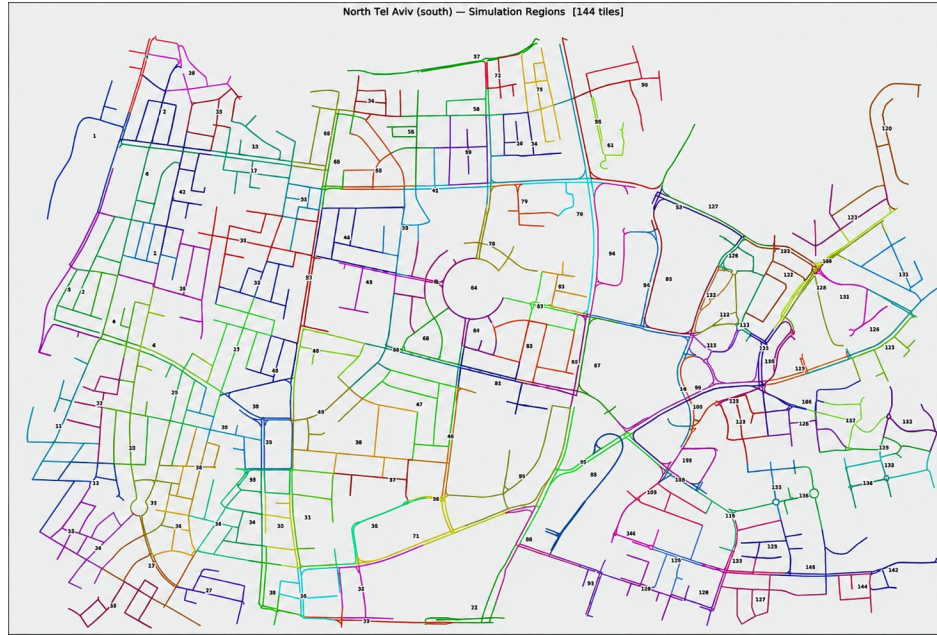


Figure 4: The northern Tel Aviv network divided into 144 simulation regions (tiles). Each region seeds an independent traffic scene, and batch runs across them produce the varied corpus of normal behavior from which the anomaly baseline is learned.

3.5 Simulation outputs and workflow

The simulator's inputs are a scenario file (a `.sim.json` describing the map, the vehicles and their routes, and the placement and parameters of every sensor) and a small set of global physics constants. Its outputs are an ordered event stream consumed live, plus, when recording is enabled, a set of on-disk products: per-vehicle ground-truth and estimated trajectories, a styled Excel workbook (dashboard, per-sensor sheets, Kalman and RMSE sheets, tracker diagnostics), an audit report, and per-track visualizations. The workflow is therefore: author or generate a scenario, run the simulation (optionally headless, in batch), collect the exports, and run feature extraction and anomaly scoring. This same workflow scales from a

single interactive run in the desktop editor to the hundreds of automated runs used to build the normal baseline.

3.6 Assumptions and limitations

Every simplifying assumption is documented so its impact is known and it can be relaxed later. Vehicles are points rather than extended bodies. DAS amplitude uses weight as a proxy for ground force. The fiber is assumed to run parallel to the road at a known offset, and no full optical waterfall is simulated: the chain is modeled at the level of derived features (amplitude, SNR, fiber position) rather than raw backscatter. These choices keep the simulator fast and analytically transparent while preserving the quantities the tracking and anomaly objectives depend on: realistic per-sensor uncertainty, realistic detection gaps, and realistic behavioral variability. In short, the simulator abstracts away the optics but reproduces the statistical character of the data that reaches the estimator.

4 Implementation

This chapter details how the system was implemented: its software architecture and component interactions, the complete flow of information from data generation to visualization, the separation between simulation and analytics, and the key algorithms and engineering decisions that made the system work in practice.

4.1 Software architecture

SimStudio is implemented entirely in Python. The simulator lives in a *simstudio* package whose modules each own one responsibility: *sim_core* (the simulation engine and sensor models), *bus* (the event bus), *tracking* (the cross-segment track manager), *kalman* (the fusion filter and row builders), *audit* and *export_xlsx* (the export and diagnostics products), and a Tkinter *gui* for interactive editing. The analytics layer lives in a separate *anomaly_model* package (*feature_extractor*, *cusum_detector*, and *scenario_scorer*) that consumes only the exported files. The numerical core depends only on NumPy. Pillow drives map rendering and openpyxl the Excel export, with a graceful CSV fallback when it is absent.

4.2 End-to-end information flow

Figure 5 traces a measurement from birth to anomaly score. The flow has eight stages, and understanding it is the key to understanding the system. The main takeaway is that data moves in one direction only, so each stage can be reasoned about, and replaced, in isolation.

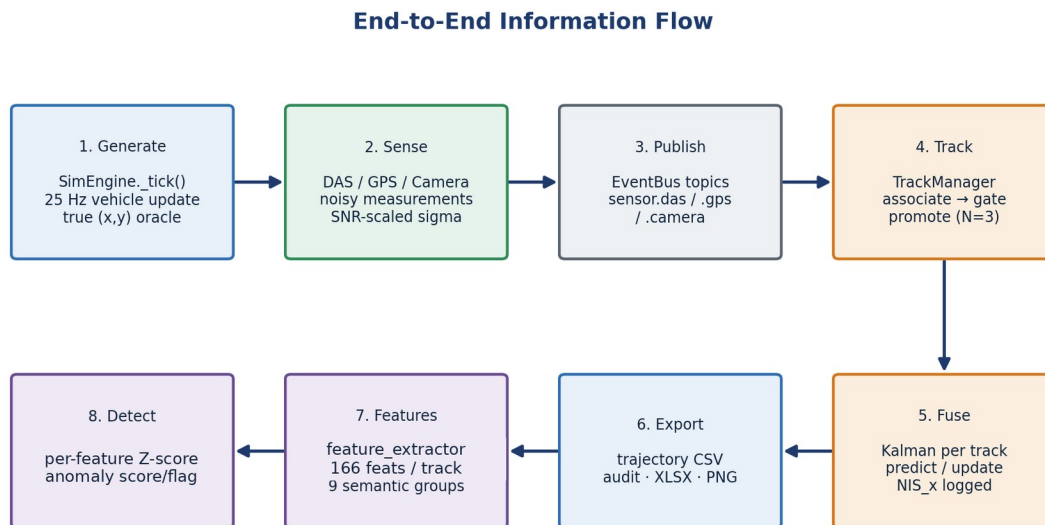


Figure 5: End-to-end information flow: generation, sensing, publication, tracking, fusion, export, feature extraction, and detection.

Generation. Every 25 Hz tick, the engine advances each vehicle's arc-length position along its lane, handles segment-to-segment transitions through the road graph, applies the lateral drift, and computes the

vehicle's true world position. That true position is published as the ground-truth oracle, used later only for error measurement, never for estimation.

Sensing and publication. In the same tick, each sensor decides whether to fire and, if so, computes its measurement and physically derived uncertainty and publishes a typed event (`sensor.das`, `sensor.gps`, or `sensor.camera`) onto the event bus. Missed DAS detections publish a lightweight miss event instead, preserving an honest record of coverage gaps.

Tracking. The track manager consumes the sensor events and assigns each to a stable global track. Association is purely kinematic: the oracle vehicle identity in the payload is treated strictly as ground truth and is never used to associate. Each candidate track is scored by predicted-versus-measured distance plus speed, heading, and segment-adjacency penalties, inside spatial and temporal gates, and the lowest-cost track wins. New tracks are born tentative and are promoted to confirmed only after three consecutive associated measurements, which suppresses spurious single-hit DAS ghosts.

Fusion. The Kalman filter runs as a post-tracking stage, replaying the same event stream grouped by track. For each track it alternates constant-acceleration prediction with measurement updates whose noise covariance R is set from the sensor's forwarded σ and further scaled by its confidence. All updates use the numerically stable Joseph form. After every update the filter logs the pre-update innovation and its variance, producing the Normalized Innovation Squared signal that the anomaly layer later consumes.

Export, feature extraction, and detection. Collected events are written to trajectories, the Excel workbook, the audit report, and per-track visualizations. The feature extractor reduces each vehicle's thousands of time-step rows to a single fingerprint of 166 features across nine semantic groups, and the scorer compares each fingerprint against the learned normal baseline, and a supplementary CUSUM diagnostic can locate the onset of any inconsistency in time.

4.3 Communication and dependencies between modules

All inter-module communication flows through the event bus, which keeps dependencies acyclic and one-directional. The simulator depends on nothing downstream. The tracker depends only on the bus event schemas. The Kalman builder depends on the tracker's identity assignment. The export and audit modules depend on the fused rows, and the analytics package depends only on the files on disk. This strict layering was enforced as an explicit engineering rule and verified by tests, because it guarantees the read-only property of the analytics layer and lets each stage be replaced independently.

4.4 Ground-truth separation and data integrity

Because the project is based on simulation, it is essential to show that the anomaly model does not benefit from information a real deployment would not have. The anomaly-detection pipeline is strictly separated from the simulator's ground-truth generation layer. During scoring, the model does not consume oracle vehicle identity, injected anomaly labels, or ground-truth trajectories. It operates only on sensor-derived measurements and the quantities computed from them. Ground truth is used solely for evaluation: computing tracking RMSE and verifying whether the injected anomalous vehicle was correctly detected. This separation ensures that the reported results reflect observable, sensor-derived behavior rather than

leakage from the simulator, and it is what allows the same pipeline to be applied unchanged to recorded real data.

4.5 Key algorithms and engineering decisions

SNR-driven dynamic measurement noise. Rather than hand-tuned constants, each sensor's uncertainty is computed from its physics at every measurement. For DAS the position standard deviation shrinks as the SNR improves and is then scaled by the measurement confidence before entering the filter as the noise covariance (Equation 7):

$$\sigma_{DAS} = \max\left(0.8, \frac{k_{DAS}}{\sqrt{SNR}}\right), \quad R = \frac{\sigma^2}{conf} \quad (7)$$

This means the filter trusts a heavy, nearby vehicle far more than a light, distant one, automatically and without manual tuning. The decision is what makes complementary fusion work: across DAS coverage gaps the filter relies on GPS and camera, and where DAS is strong it dominates by design (its measurement variance can be an order of magnitude smaller than that of GPS).

The ghost-track association fix. Early testing revealed an instructive failure: a well-tracked vehicle could acquire zero GPS measurements. High-rate DAS hits that fell just outside the gate spawned tentative ghost tracks with the same kinematics as the true track. When a sparse GPS fix arrived, the ambiguity guard saw two near-identical candidates and, rather than risk a wrong association, opened a new fragment track, orphaning the GPS data. The fix was to make the ambiguity guard identity-aware: only a second confirmed track can block an association, so transient ghosts are ignored and GPS flows into the correct confirmed track. The broader lesson is that in multi-rate sensor fusion, association logic must distinguish established tracks from provisional ones.

Innovation logging as the anomaly substrate. Logging the filter's innovation statistics turns the Kalman filter into a sensor of its own consistency. The Normalized Innovation Squared in the x-direction is the ratio of the squared pre-update innovation to its variance (Equation 8):

$$NIS_x = \frac{v_x^2}{S_{xx}} \quad (8)$$

Under a consistent, well-tuned filter this quantity follows a $\chi^2(1)$ distribution with an expected value of one. A sustained rise indicates that reality has diverged from the model, which is the signature of an anomaly. Routing this signal into the CUSUM detector provides a principled, model-based temporal anomaly indicator.

Backward-compatible, test-guarded evolution. The pipeline grew through versioned stages, each preserving exact behavior on earlier scenarios. When the tracker was inserted in front of the Kalman filter, the one-to-one identity case was kept byte-identical so that no accuracy regression could pass unnoticed. A test suite spanning the simulator core, the Kalman filter, the tracker, and the export pipeline guards every signal the anomaly model depends on.

4.6 The anomaly model in detail

Figure 6 summarizes the anomaly-detection architecture. The model is realized in two layers, plus a supplementary temporal diagnostic. **Layer 1: feature extraction** turns each vehicle's trajectory into a single 166-number fingerprint. **Supplementary diagnostic: CUSUM** watches the Kalman NIS over time and localizes when a deviation began. **Layer 2: the scenario scorer** learns the normal fingerprint distribution and scores each vehicle by deviation.

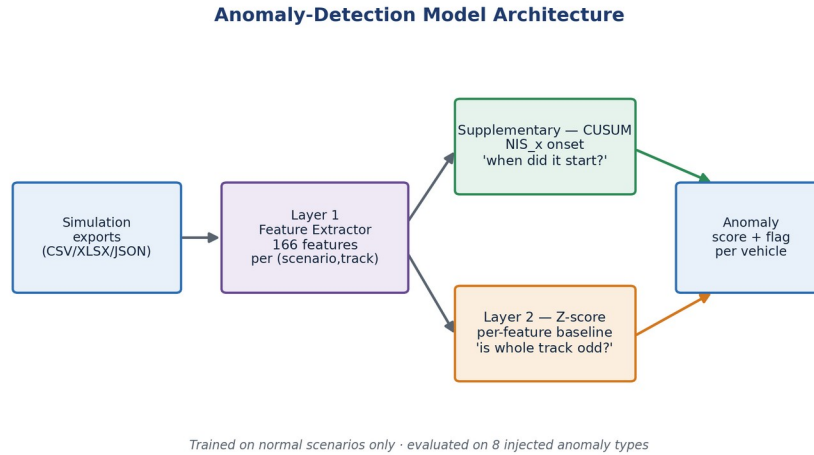


Figure 6: Anomaly-detection model architecture: feature extraction feeds the trajectory-level Z-score scorer, which produces the per-vehicle anomaly score and flag, while a supplementary CUSUM diagnostic provides temporal localization.

Terminology and roles. To avoid ambiguity, the relationship between the model components is stated explicitly. The reported anomaly results are produced by the interpretable per-feature Z-score baseline, the model's only trajectory-level classifier. CUSUM is a supplementary diagnostic: it provides temporal localization of abnormal behavior rather than classification [6]. The 166 features are organized into the groups shown in Table 2.

Feature group	Examples	Purpose
Kinematics	speed, acceleration, jerk	Detect speeding, braking, reckless behavior
Kalman quality	innovation, NIS, RMSE proxy	Detect prediction/measurement inconsistency
Sensor availability	DAS / GPS / camera coverage	Detect sensing gaps or unusual visibility
DAS signal quality	SNR, amplitude, estimated weight	Detect overweight or unusual DAS response
Spacing	distance to nearby vehicles	Detect tailgating / collision-related behavior
Lateral behavior	lane offset, lateral variance	Detect weaving / straddling

Table 2: Feature groups produced by the feature extractor (166 features per vehicle in total).

The temporal layer uses Page's one-sided CUSUM, which accumulates the evidence that the current NIS exceeds a reference level k , and raises an alarm when the running statistic crosses a threshold h (Equation 9):

$$S_t = \max\left(0, S_{t-1} + \left(NIS_x - k\right)\right), \quad \text{alarm if } S_t \geq h \quad (9)$$

The trajectory-level detector standardizes each feature against the learned normal mean and standard deviation and flags a vehicle when its largest absolute Z-score exceeds the decision threshold (Equation 10):

$$z_i = \frac{x_i - \mu_i}{\sigma_i}, \quad \text{score} = \max_i |z_i| \quad (10)$$

4.7 Repository structure

The deliverables are organized so that the simulator, the analytics layer, the maps, and the recorded outputs each occupy a clearly separated part of the tree. The abridged layout below highlights the components discussed above and the strict simulator / analytics separation that underpins the read-only design.

```

final-project/
├── src/simstudio/          # Simulation engine + desktop editor
│   ├── sim_core.py        # 25 Hz engine · vehicle physics · sensor models
│   ├── bus.py             # EventBus (publish / subscribe)
│   ├── tracking.py        # Cross-segment TrackManager (identity)
│   ├── kalman.py          # 6-state CA fusion + row builders
│   ├── audit.py           # Trajectory / audit exporter
│   ├── export_xlsx.py     # Styled Excel workbook + charts
│   └── gui/app.py         # Tkinter editor · geometry · models · config
├── anomaly_model/         # Read-only analytics (never imports simstudio)
│   ├── feature_extractor.py # Trajectory → 166-feature fingerprint
│   ├── cusum_detector.py  # Temporal NIS_x onset detector
│   ├── scripts/scenario_scorer.py # Per-feature Z-score scoring (numpy-only)
│   ├── simulations/       # Normal + 9 injected-anomaly scenarios
│   └── outputs/           # Feature tables · models · scores
├── maps/                  # OpenStreetMap networks (Florentin · N. Tel Aviv)
├── outputs/               # Trajectories · audit reports · PNGs · workbooks
├── tests/                 # Pytest suite (sim · kalman · tracker · export)
├── scripts/run_app.py     # Application entry point
├── README.md · CHANGELOG.md · pyproject.toml · install.sh

```

5 Analysis of Results

Results are organized around the project's evaluation questions and reported against the quantitative targets fixed in the work plan. Tracking and fusion accuracy are examined first, followed by sensor-fusion behavior, anomaly-detection performance, a failure analysis of the missed anomalies, and a consolidated comparison with the targets.

5.1 Tracking and fusion accuracy

Fusion accuracy was measured against the simulator's ground-truth trajectories using the position RMSE (Equation 11):

$$RMSE = \sqrt{\frac{1}{N} \sum_k \left((\hat{x}_k - x_k)^2 + (\hat{y}_k - y_k)^2 \right)} \quad (11)$$

In a representative cross-segment run, a vehicle tracked across the seg1→seg2 boundary while fusing all three sensors achieved a position **RMSE of 0.49 m with a maximum instantaneous error of 1.52 m** (Figure 7). The estimate follows the ground truth throughout, including across the segment transition. GPS fixes appear as green markers that gently correct the estimate between DAS coverage zones, as the fusion design intends. Across the nine anomaly-evaluation scenarios the mean per-track RMSE was **0.35 m** (best 0.19 m, worst 0.45 m), consistently sub-meter and an order of magnitude inside the ± 5 m tolerance implied by the position and speed targets.

XY spatial trajectory — global_track_id = T000001

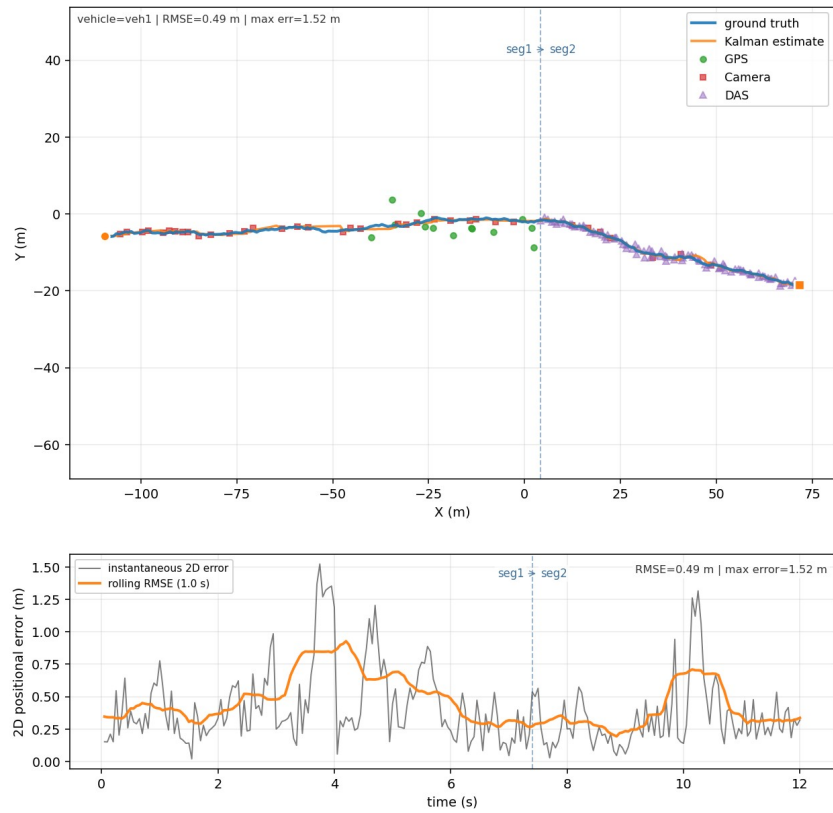


Figure 7: Fused multi-sensor track versus ground truth for a cross-segment vehicle. Top: spatial trajectory with DAS, camera, and GPS measurements, and the dashed line marks the segment boundary. Bottom: instantaneous and rolling 2-D error (RMSE = 0.49 m, max 1.52 m).

The same behavior holds over longer routes. Figure 8 shows a vehicle tracked continuously across four connected segments in the over-speeding scenario, retaining a single identity through both transitions at **0.37 m RMSE (max 1.11 m)**, confirming that the tracker scales beyond a single hand-off without fragmenting the trajectory.

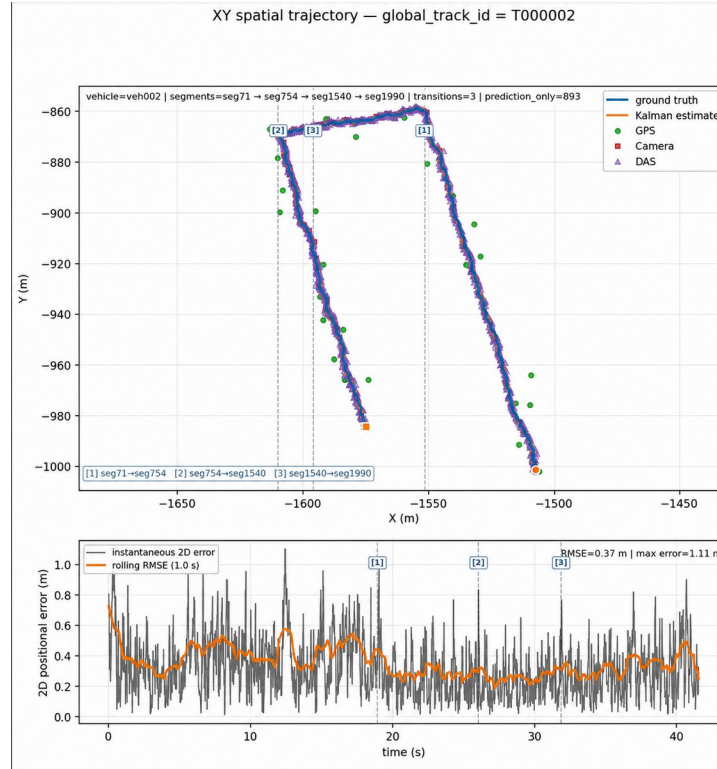


Figure 8: Continuous tracking across four connected segments (over-speeding scenario). Top: spatial trajectory with DAS, camera, and GPS measurements and two segment transitions. Bottom: 2-D error over time (RMSE = 0.37 m, max 1.11 m).

5.2 Sensor contribution and fusion behavior

The accuracy above is a direct consequence of two design choices. The identity-aware association fix ensured that GPS measurements reached the correct confirmed track rather than orphan fragments, and the SNR-driven noise model let the filter weight each sensor by its physically derived reliability. In practice the three modalities play distinct roles. DAS provides the dense, high-rate backbone of the trajectory, the camera contributes accurate fixes at intersections, and GPS supplies low-rate absolute corrections that bound long-term drift between DAS coverage zones. Because DAS measurement variance can be roughly an order of magnitude smaller than that of GPS, the filter leans on DAS where it is strong and gracefully falls back to the other sensors where DAS is sparse, which is the intended complementary-fusion behavior.

5.3 Anomaly-detection performance

The anomaly model was trained on the behavioral fingerprints of roughly 150 normal scenarios, over fourteen thousand vehicle-tracks spanning twelve deliberately varied normal conditions. It was then evaluated on nine scenarios, each containing one injected anomalous vehicle drawn from eight anomaly categories (Table 3).

Anomaly scenario	Injected vehicle score (σ)	Detected?
Hard braking	383.5	Yes
Stalled vehicle	376.1	Yes
Collision	315.6	Yes

Anomaly scenario	Injected vehicle score (σ)	Detected?
Reckless driver	25.1	Yes
Speeding	21.6	Yes
Overweight vehicle	19.0	Yes
Lane straddling	6.2	No
Lane weaving	5.0	No
Tailgating	2.1	No

Table 3: Anomaly-detection results across nine injected scenarios. Detection threshold $\approx 15.4 \sigma$. The injected vehicle was correctly flagged in six of nine cases.

How the threshold was chosen. The decision threshold ($\approx 15.4 \sigma$) was selected from the normal baseline distribution in order to keep the false-positive rate low. A vehicle is flagged as anomalous only when its maximum per-feature deviation exceeds this conservative threshold, which sits well above the spread of normal vehicles and therefore favors precision over recall.

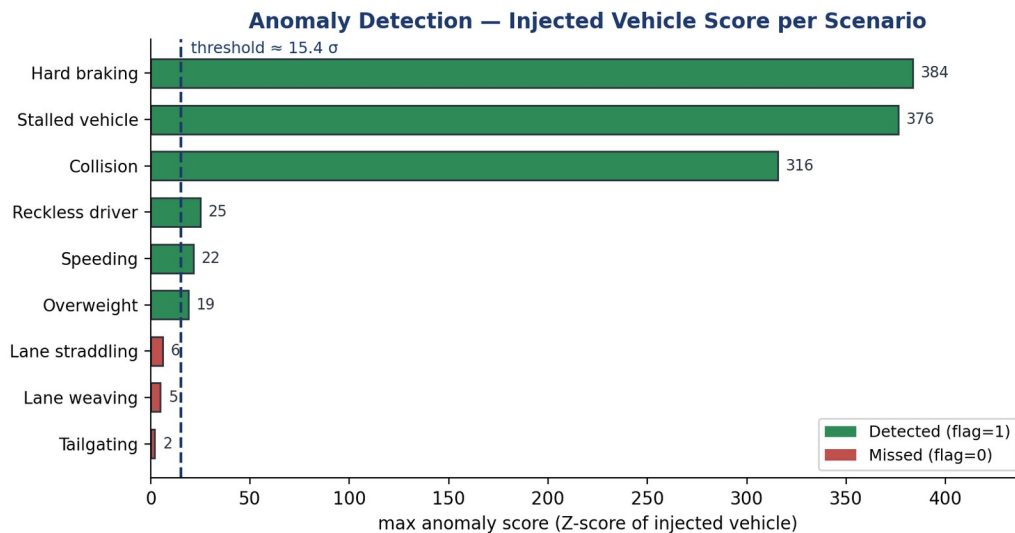


Figure 9: Anomaly score of the injected vehicle per scenario, against the $\approx 15.4 \sigma$ decision threshold. Longitudinal and sensor-level anomalies separate clearly from the normal baseline, while subtle lateral and spacing anomalies do not.

The model correctly flagged the injected vehicle in **six of the nine scenarios** (speeding, hard braking, stalled vehicle, collision, reckless driving, and overweight vehicle), meeting the work-plan target of detecting at least five of ten anomalies. The separation is clear for the detected classes: scores of hundreds of sigma for braking, stalls, and collisions, and roughly 19–25 σ for the more subtle speeding, reckless, and overweight cases, all above the threshold while normal background vehicles remain low. In the stalled-vehicle and collision scenarios the model also flagged the vehicles immediately behind the event, a physically correct response, since those vehicles are forced to stop abnormally as well.

Examining a single scenario shows how a decision is reached. In the over-speeding case (Figure 10) the injected vehicle T000001 stands alone at 21.6 σ while the three background vehicles sit near 2 σ , well inside the normal band.

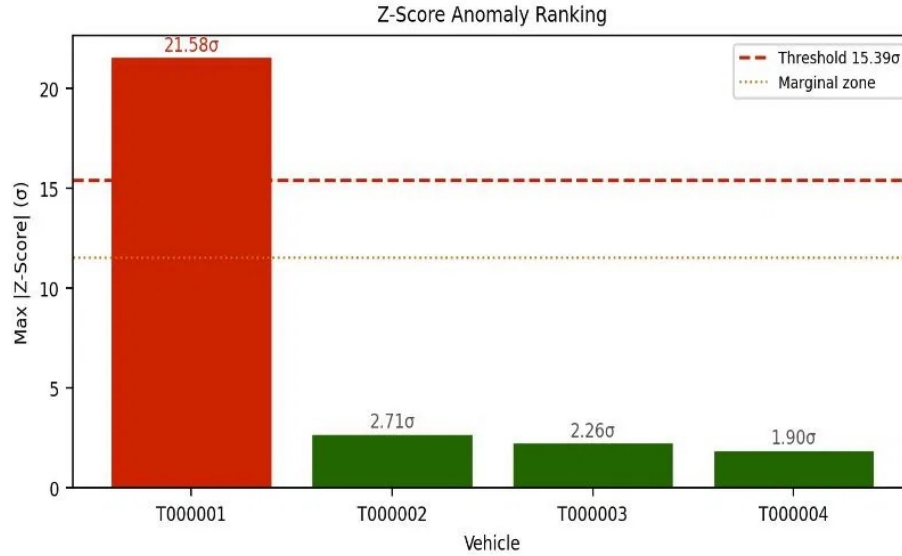


Figure 10: Per-vehicle Z-score ranking for the over-speeding scenario. The injected vehicle (red) exceeds the decision threshold by a wide margin, while background vehicles (green) remain in the normal band.

The model is also interpretable: it reports which features drove each flag. Figure 11 overlays the injected vehicle's measured feature values on the normal training distributions, showing that its relative speed at closest approach, mean speed over the run, and lateral velocity all fall outside the normal range, the expected signature of over-speeding combined with aggressive lane use.

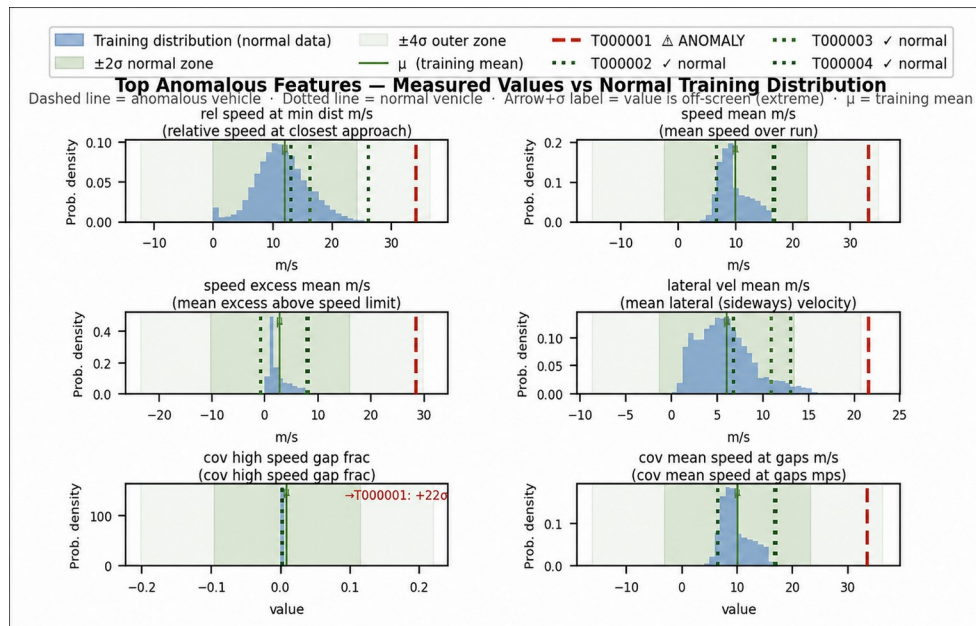


Figure 11: Top anomalous features for the injected vehicle versus the normal training distribution. Dashed lines mark the $\pm 4\sigma$ envelope, and the vehicle's values (red) lie outside it on speed and lateral-velocity features, explaining the flag.

5.4 Missed anomalies and failure analysis

The model performs best on longitudinal and sensor-level anomalies (hard braking, stalled vehicles, collisions, speeding, reckless driving, and overweight vehicles) because these create strong deviations in speed, acceleration, Kalman consistency, or DAS amplitude. It performs less well on subtle lateral or interaction-based anomalies, namely lane weaving, lane straddling, and tailgating, which scored 5.0σ , 6.2σ , and 2.1σ respectively and therefore fell below the conservative decision threshold.

This outcome is informative rather than a simple shortcoming. These behaviors remain close to the normal-driving distribution under the current feature set, especially because the threshold is chosen to avoid false alarms. The result directly identifies the next research direction: adding stronger lateral-dynamics and inter-vehicle-spacing features, fusing the CUSUM temporal signal into the trajectory score, and training a supervised classifier once enough labeled anomaly scenarios are available. In this sense the missed detections map out exactly where additional signal and labeled data would yield the largest improvement.

5.5 Comparison with quantitative targets

Table 4 consolidates the achieved results against the work-plan targets.

Target (work plan)	Goal	Achieved	Status
Cross-street tracking accuracy	$\geq 90\%$	Sub-meter RMSE (0.35 m mean, 0.49 m showcase) across segment boundaries	Met
Anomaly detection	$\geq 5 / 10$	6 of 9 injected anomalies detected	Met
Kalman fusion success	$\geq 90\%$	Identity-aware association, confirmed-track fusion of all 3 sensors	Met
Position / speed tolerance	$\leq \pm 5$ m	Max error 1.52 m, mean RMSE 0.35 m	Met
Simultaneous vehicles / segments	$\leq 10 / \leq 5$	Multi-vehicle, multi-segment scenarios run routinely	Met
End-to-end latency	≤ 60 s	Offline batch pipeline, real-time profiling pending	Partial

Table 4: Quantitative performance targets and achieved results.

Of the work-plan targets, the two primary objectives and the supporting accuracy targets were met. Only end-to-end real-time latency remains to be profiled, as evaluation was conducted in an offline batch configuration suited to algorithm development.

5.6 Real-world validation: the Klausner Street experiment

While the preceding sections evaluate the system in simulation, the project also included a real-world experiment intended to begin closing the loop to physical data. In collaboration with a research colleague, a camera and a distributed-acoustic-sensing (DAS) channel were operated on an existing telecommunication fiber along Klausner Street, beside the Tel Aviv University campus (Figure 12). The recording captured everyday urban traffic, including a clearly identifiable signature, a city bus decelerating to a halt at a bus stop, which was isolated from the raw DAS trace as event #4 and used as the reference for this validation.

The purpose of the experiment is to exercise the full tracking-and-fusion pipeline on data whose timing, geometry, and noise originate in the physical world rather than in the simulator. Because dedicated DAS-interrogator hardware and a fully instrumented street were not available for the duration of the project, this single event is a partial but genuinely real validation of the approach.

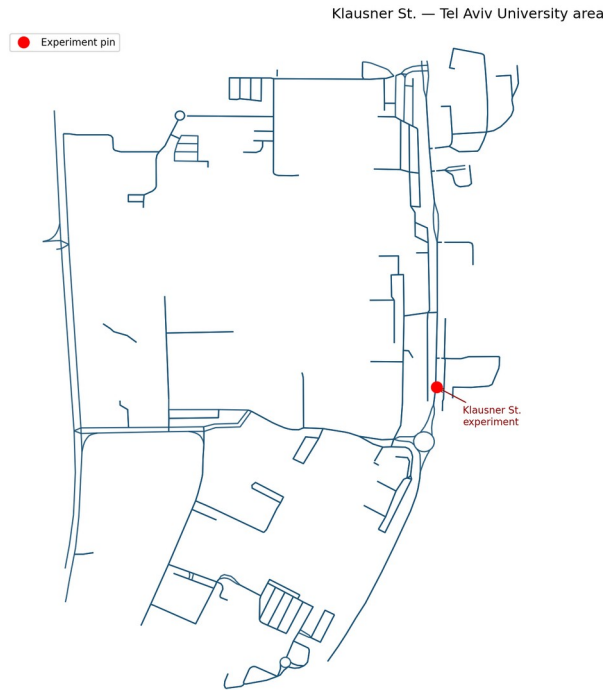


Figure 12: Location of the Klausner Street experiment beside the Tel Aviv University campus. The red pin marks the camera + DAS measurement point on the real street network imported from OpenStreetMap.

The recorded event was processed through the exact pipeline used in simulation: DAS event #4 combined with a co-located camera (cam2, reported $\sigma \approx 0.2$ m), replayed on the true Klausner Street geometry imported from OpenStreetMap. The filter fused 84 DAS and 30 camera measurements over a 41.8 s track and held the bus's trajectory to a position RMSE of 1.17 m against ground truth, well within the project's ± 5 m tolerance, while running on prediction alone for 42% of the run, including intervals of up to 4.0 s without any measurement, and never losing the track through the complete stop at the bus stop.

The per-sensor breakdown is instructive. The DAS channel was highly accurate (RMSE 0.33 m, bias ≈ 0), while the camera channel's measured error (RMSE 2.49 m) far exceeded its reported uncertainty. This is consistent with the report's central finding: DAS is the dependable backbone of the fusion, and an optical sensor's uncertainty must be modeled honestly for its measurements to help rather than hurt.

Quantity	Real DAS event #4 + camera
Sensors fused	DAS + camera (84 + 30 meas.)
Track position RMSE vs. ground truth	1.17 m
DAS per-sensor error (RMSE)	0.33 m (bias $\approx$ 0)
Camera per-sensor error (RMSE)	2.49 m
Prediction-only rows	42.1%
Longest gap without a measurement	3.98 s
Track duration	41.8 s

Table 5: Klausner Street experiment, fusing the real DAS recording with a co-located camera. Per-sensor errors are measured against ground truth.

The experiment shows that the pipeline runs unchanged on real recorded data and tracks a heavy vehicle through a complete stop at meter-level accuracy from sparse real measurements. It also surfaces exactly the failure mode the design anticipates, an optical sensor reporting tighter uncertainty than it deserves, and points to per-deployment camera calibration as the first step before a larger field campaign. Even from a single event, the experiment validates the simulation-to-reality bridge and sets a concrete agenda for the hardware validation discussed in Chapter 6.

6 Conclusions and Further Work

6.1 Results against the original goals

The project set two primary objectives in the Introduction, and the results map directly onto them. **(1) Reliable cross-segment vehicle tracking** was achieved at sub-meter accuracy (0.49 m RMSE on a representative fused track and 0.35 m on average), with a single identity maintained across segment transitions. **(2) Multi-sensor fusion** of DAS, camera, and GPS was realized through physically derived, SNR-driven measurement uncertainty, allowing each sensor to contribute according to its reliability. **(3) Anomaly detection** from sensor-derived data flagged the injected anomalous vehicle in six of nine scenarios, meeting the at-least-five-of-ten target. Underpinning all three is a reproducible simulation environment that models the DAS measurement chain from vehicle weight to Kalman update, and a strictly read-only analytics layer that mirrors how a real deployment would operate on recorded data.

6.2 Suggestions to improve performance

The clearest opportunity is the three missed anomaly classes. With labeled anomaly scenarios now available, training a supervised classifier (such as a Random Forest) should improve sensitivity to subtle lateral and spacing anomalies that the unsupervised baseline cannot separate from normal variability. Adding explicit lateral-dynamics and inter-vehicle-spacing features, and fusing the CUSUM temporal signal directly into the trajectory score, would further raise recall while keeping the false-positive rate low. On the tracking side, replacing the kinematic association cost with a Mahalanobis-gated, covariance-aware association would improve robustness in dense traffic.

6.3 Future development and research

Three directions stand out. **First**, closing the loop to hardware: building on the single-event Klausner Street validation (Section 5.6) toward a sustained field campaign, validating the simulator's physical models against real DAS recordings on an instrumented street, and profiling the pipeline in a streaming configuration to confirm the end-to-end latency target. **Second**, deepening the DAS physics by simulating the raw optical waterfall and signal-processing front end, so that event detection can be developed directly on channel data [2]. **Third**, scaling from neighborhoods to a city: extending cross-segment tracking to cross-street hand-off across many intersections, and surfacing fused trajectories and anomaly alerts on a live monitoring dashboard. Together these would carry the work from a validated simulation framework toward a deployable urban sensing system.

7 Project Documentation

All project deliverables are documented and version-controlled. This report describes them rather than reproducing code or data sheets.

7.1 Software

The software comprises two packages. The *simstudio* package contains the simulation engine, sensor models, event bus, track manager, Kalman fusion, export and audit modules, and the Tkinter desktop editor. It ships with an installable project definition, an install script, and a packaged macOS application. The *anomaly_model* package contains the feature extractor, the CUSUM detector, the scenario scorer (per-feature Z-score), and the batch extraction scripts. A test suite spanning the simulator core, Kalman filter, tracker, and export pipeline accompanies the code, together with a README, a changelog, a physical-models reference, and a validation protocol.

7.2 Simulations and datasets

The deliverables include the imported OpenStreetMap road networks (Florentin and northern Tel Aviv) with their sensor placements, roughly 150 normal-traffic scenarios spanning twelve conditions, and nine injected-anomaly scenarios covering eight categories. For each run the system produces trajectories, a styled Excel workbook, an audit report, and per-track visualizations, and the anomaly layer produces per-vehicle feature tables and scored output files.

7.3 Documentation contents and location

Project documentation includes the user-facing README and install guide, the physical-models and equations references, the changelog tracking the versioned evolution of the pipeline, the validation protocol, and an anomaly-interpretation guide. The complete code base, documentation, simulation scenarios, and result datasets are maintained in the project GitHub repository: <https://github.com/noasapir3/simstudio-das-fusion>

8 References

- [1] L. Hen, E. Lichtenstadt, K. Cohen, and A. Lellouch, “Enhanced Traffic Monitoring using a Convolutional Neural Network with Video Signal as a Labels Producer over Fiber-Optic Seismology,” research report, School of Electrical Engineering, Tel Aviv University, 2023.
- [2] K. Cohen, L. Hen, and A. Lellouch, “A fiber-optic traffic monitoring network trained with video inputs,” Scientific Reports, vol. 15, art. 14928, 2025. Preprint: “Training a Distributed Acoustic Sensing Traffic Monitoring Network With Video Inputs,” arXiv:2412.12743, 2024.
- [3] “Exploring Cost-Effective Traffic Monitoring Through Kalman-Filter-Based Fusion of Fiber-Optic Seismology and Computer-Vision Tracking Data,” project report, School of Electrical Engineering, Tel Aviv University, 2024.
- [4] N. Nissan and O. Nissan, “Automatic Calibration of DAS and Camera,” final engineering project report, School of Electrical Engineering, Tel Aviv University, 2025.
- [5] R. E. Kalman, “A New Approach to Linear Filtering and Prediction Problems,” Journal of Basic Engineering, vol. 82, no. 1, pp. 35–45, 1960.
- [6] E. S. Page, “Continuous Inspection Schemes,” Biometrika, vol. 41, no. 1–2, pp. 100–115, 1954.
- [7] G. Welch and G. Bishop, “An Introduction to the Kalman Filter,” University of North Carolina at Chapel Hill. <http://www.cs.unc.edu/~welch/kalman/>
- [8] OpenStreetMap contributors, OpenStreetMap. <https://www.openstreetmap.org>

Appendix A: Feature Catalog

This appendix lists the 156 features used by the anomaly-detection model, exactly the set of columns the trained scorer consumes. For each feature the table gives a description of how it is computed, its expected output range, and the data source from which it is collected. The features are grouped to match the semantic groups of Table 2. Identifier and metadata fields that the extractor also writes but that the model does not score (for example scenario and track identifiers) are intentionally excluded.

A.1 Scenario context (1 features)

Feature	Description (how it is computed)	Output range	Source
sc_speed_limit_mps	Min speed limit across segments [m/s]	≥ 0 m/s	Scenario JSON (vehicle / sensor config)

A.2 Kinematics (35 features)

Feature	Description (how it is computed)	Output range	Source
kin_speed_mean_mps	Mean estimated speed [m/s]	0–40 m/s	Trajectory CSV (Kalman estimates)
kin_speed_max_mps	Max estimated speed [m/s]	0–50+ m/s	Trajectory CSV (Kalman estimates)
kin_speed_min_mps	Minimum estimated speed [m/s], near 0 = vehicle stopped or stalled	0–40 m/s	Trajectory CSV (Kalman estimates)
kin_speed_std_mps	Std dev of estimated speed [m/s]	0–15 m/s	Trajectory CSV (Kalman estimates)
kin_speed_p90_mps	90th-percentile estimated speed [m/s], robust peak speed proxy	0–50+ m/s	Trajectory CSV (Kalman estimates)
kin_speed_over_limit_frac	Fraction of timesteps where speed > speed_limit $\times 1.05$	0–1	Trajectory CSV (Kalman estimates)
kin_speed_excess_max_mps	Max speed excess above speed limit on over-limit timesteps [m/s]	0–30+ m/s	Trajectory CSV (Kalman estimates)
kin_speed_excess_mean_mps	Mean speed excess above speed limit on over-limit timesteps [m/s]	0–20 m/s	Trajectory CSV (Kalman estimates)
kin_speed_cv	Coefficient of variation of speed (std/mean), high = erratic speed profile	0– ∞ (ratio)	Trajectory CSV (Kalman estimates)
kin_lateral_vel_mean_mps	Mean $ v_{y_hat} $ [m/s], average lateral velocity component from Kalman filter, sustained high value = persistent sideways motion	0–5+ m/s	Trajectory CSV (Kalman estimates)
kin_lateral_vel_max_mps	Max $ v_{y_hat} $ [m/s], peak lateral speed from Kalman filter, high = sudden sideways lurch (weaving onset, lane-change, collision side-swipe)	0–10+ m/s	Trajectory CSV (Kalman estimates)
kin_speed_jump_max_mps	Max $ \Delta \text{speed} $ between consecutive 50 Hz ticks [m/s], sudden jump exceeding physical limits = ghost track, sensor outlier, or emergency stop	0–20+ m/s	Trajectory CSV (Kalman estimates)
kin_heading_change_rate_max_rad_per_s	Max $ \Delta \text{heading} / \Delta t $ [rad/s] while speed > 0.5 m/s, high = abrupt swerve, tight curve, or lane-change manoeuvre	0–5+ rad/s	Trajectory CSV (Kalman estimates)
kin_stopped_frac	Fraction of timesteps where estimated speed < 0.5 m/s (stopped)	0–1	Trajectory CSV (Kalman estimates)
kin_stop_event_count	Number of distinct stop events (transitions into speed < 0.5 m/s)	0–n	Trajectory CSV (Kalman estimates)
kin_max_stopped_steps	Maximum consecutive timesteps where speed < 0.5 m/s	0–n	Trajectory CSV (Kalman estimates)
kin_max_stopped_duration_s	Longest continuous stop duration [s]	0– ∞ s	Trajectory CSV (Kalman estimates)
kin_accel_mean_abs_mps2	Mean absolute acceleration magnitude [m/s ²]	0–5 m/s ²	Trajectory CSV (Kalman estimates)
kin_accel_max_abs_mps2	Max absolute acceleration magnitude [m/s ²]	0–10+ m/s ²	Trajectory CSV (Kalman estimates)
kin_accel_std_mps2	Std dev of acceleration magnitude [m/s ²]	0–5 m/s ²	Trajectory CSV (Kalman estimates)
kin_phys_impossible_v	Row count where estimated speed > 40 m/s (physically suspect)	0–n	Trajectory CSV (Kalman estimates)
kin_phys_impossible_a	Row count where estimated $ \text{accel} > 8$ m/s ² (physically suspect)	0–n	Trajectory CSV (Kalman estimates)
kin_lateral_accel_max_mps2	Max $ a_{y_hat} $ [m/s ²], peak lateral acceleration from Kalman filter, large value = aggressive steering / weaving manoeuvre / side-impact	0–10+ m/s ²	Trajectory CSV (Kalman estimates)
kin_decel_max_mps2	Maximum deceleration (most negative accel) [m/s ²]	0–10+ m/s ²	Trajectory CSV (Kalman estimates)
kin_high_decel_frac	Fraction of timesteps where deceleration > 3 m/s ²	0–1	Trajectory CSV (Kalman estimates)
kin_decel_event_count	Number of distinct hard-braking events (decel > 3 m/s ²)	0–n	Trajectory CSV (Kalman estimates)
kin_jerk_max_mps3	Maximum jerk (rate of change of acceleration) [m/s ³]	0–20+ m/s ³	Trajectory CSV (Kalman estimates)
kin_jerk_mean_abs_mps3	Mean absolute jerk [m/s ³], high = aggressive/erratic driving	0–10 m/s ³	Trajectory CSV (Kalman estimates)

kin_lateral_dev_mean_m	Mean lateral deviation of y_{hat} from its track mean [m]	0–5 m	Trajectory CSV (Kalman estimates)
kin_lateral_dev_max_m	Max lateral deviation of y_{hat} from track mean [m]	0–10 m	Trajectory CSV (Kalman estimates)
kin_lateral_dev_std_m	Std dev of lateral deviation [m]	0–5 m	Trajectory CSV (Kalman estimates)
kin_lateral_peak_to_peak_m	Peak-to-peak lateral range of lane-relative offset [m], high = wide weaving	0–6+ m	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)
kin_lateral_oscillation_ratio	$\text{std}(\text{lat_dev}) / \text{mean}(\text{lat_dev})$, high value indicates lane weaving	0–5+ (ratio)	Trajectory CSV (Kalman estimates)
kin_lateral_speed_max_mps	Max rate of lateral position change $ \Delta y_{\text{hat}} / \Delta t $ [m/s], captures rapid lane-change events	0–5+ m/s	Trajectory CSV (Kalman estimates)
kin_road_heading_diff_mean_rad	Mean absolute angular difference between vehicle heading (from v_y/v_x) and road segment design heading [rad], high = crossing lane or wrong-way driving, requires scenario JSON points array	0– π rad	Trajectory CSV (Kalman estimates) + Scenario JSON (vehicle / sensor config)

A.3 Kalman filter quality (13 features)

Feature	Description (how it is computed)	Output range	Source
kf_pos_err_mean_m	Mean position error vs ground truth [m]	0–20 m	Trajectory CSV (Kalman estimates)
kf_pos_err_max_m	Max position error [m]	0–50+ m	Trajectory CSV (Kalman estimates)
kf_pos_err_std_m	Std dev of position error [m]	0–15 m	Trajectory CSV (Kalman estimates)
kf_pos_err_p90_m	90th percentile position error [m]	0–30 m	Trajectory CSV (Kalman estimates)
kf_track_rmse_m	Overall track RMSE [m]	0–20 m	Trajectory CSV (Kalman estimates)
kf_rolling_rmse_spike_count	Count of 10-step windows where rolling RMSE > 2× track RMSE	0–n	Trajectory CSV (Kalman estimates)
kf_sigma_pos_mean_m	Mean Kalman position uncertainty σ [m]	0–10 m	Trajectory CSV (Kalman estimates)
kf_sigma_pos_max_m	Max Kalman position uncertainty σ [m]	0–20+ m	Trajectory CSV (Kalman estimates)
kf_consistency_ratio_mean	$\text{Mean}(\text{pos_err} / \text{sigma_pos})$, > 2 = overconfident filter	0–5+ (ratio)	Trajectory CSV (Kalman estimates)
kf_consistency_ratio_max	$\text{Max}(\text{pos_err} / \text{sigma_pos})$	0–10+ (ratio)	Trajectory CSV (Kalman estimates)
kf_overconfident_frac	Fraction of rows where $\text{pos_err} > 2 \times \text{sigma_pos}$	0–1	Trajectory CSV (Kalman estimates)
kf_sigma_growth_rate_during_gaps_m_per_s	Mean rate of sigma_pos increase [m/s] during prediction-only (no-measurement) periods, abnormally fast = filter diverging	0– ∞ m/s	Trajectory CSV (Kalman estimates)
kf_post_dropout_jump_m	Mean 2-D position jump $ \Delta x_{\text{hat}} $ at every transition from a prediction_only run back to a measurement row [m], large = filter drifted during the gap (long dropout) or ghost track that reappeared at a different position	0–50+ m	Trajectory CSV (Kalman estimates)

A.4 Coverage and sensor utility (54 features)

Feature	Description (how it is computed)	Output range	Source
cov_total_rows	Total trajectory rows (measurement + prediction)	≥ 1 (integer)	Trajectory CSV (Kalman estimates)
cov_pred_only_count	Number of prediction-only (no measurement) rows	0–n	Trajectory CSV (Kalman estimates)
cov_pred_only_frac	Fraction of rows that are prediction-only	0–1	Trajectory CSV (Kalman estimates)
cov_max_consec_pred	Max consecutive prediction-only rows (longest dropout gap)	0–n	Trajectory CSV (Kalman estimates)
cov_n_dropout_events	Number of distinct measurement dropout events	0–n	Trajectory CSV (Kalman estimates)
cov_duration_s	Track duration [s]	0– ∞ s	Trajectory CSV (Kalman estimates)
cov_das_active_frac	Fraction of track rows where DAS has a measurement	0–1	Trajectory CSV (Kalman estimates)
cov_cam_active_frac	Fraction of track rows where Camera has a measurement	0–1	Trajectory CSV (Kalman estimates)
cov_gps_active_frac	Fraction of track rows where GPS has a measurement	0–1	Trajectory CSV (Kalman estimates)
cov_multi_sensor_frac	Fraction of rows where ≥ 2 sensors are active simultaneously	0–1	Trajectory CSV (Kalman estimates)
cov_single_sensor_frac	Fraction of rows where exactly 1 sensor is active	0–1	Trajectory CSV (Kalman estimates)
cov_no_sensor_frac	Fraction of rows where no sensor provides a measurement	0–1	Trajectory CSV (Kalman estimates)
cov_das_cam_overlap_frac	Fraction of rows where both DAS and Camera are active	0–1	Trajectory CSV (Kalman estimates)
cov_das_gps_overlap_frac	Fraction of rows where both DAS and GPS are active	0–1	Trajectory CSV (Kalman estimates)
cov_cam_gps_overlap_frac	Fraction of rows where both Camera and GPS are active	0–1	Trajectory CSV (Kalman estimates)
cov_all_sensors_frac	Fraction of rows where all three sensors are active	0–1	Trajectory CSV (Kalman estimates)
cov_err_das_only_m	Mean position error on rows where DAS is the sole sensor [m]	0–20 m	Trajectory CSV (Kalman estimates)
cov_err_cam_only_m	Mean position error on rows where Camera is the sole sensor [m]	0–20 m	Trajectory CSV (Kalman estimates)
cov_err_gps_only_m	Mean position error on rows where GPS is the sole sensor [m]	0–20 m	Trajectory CSV (Kalman estimates)
cov_err_multi_sensor_m	Mean position error on rows where ≥ 2 sensors are active [m]	0–10 m	Trajectory CSV (Kalman estimates)

cov_err_no_sensor_m	Mean position error on prediction-only rows [m]	0–30+ m	Trajectory CSV (Kalman estimates)
cov_fusion_benefit_vs_best_single	best_single_err – multi_sensor_err [m], positive = fusion helps	–∞ to +∞ m	Trajectory CSV (Kalman estimates)
cov_err_cam_no_das_m	Mean position error when Camera active but DAS absent [m]	0–20 m	Trajectory CSV (Kalman estimates)
cov_err_cam_with_das_m	Mean position error when Camera and DAS both active [m]	0–10 m	Trajectory CSV (Kalman estimates)
cov_das_fusion_benefit_m	err_cam_no_das – err_cam_with_das [m]	–∞ to +∞ m	Trajectory CSV (Kalman estimates)
cov_gap_frac_early	Fraction of the first third of the track that is prediction-only	0–1	Trajectory CSV (Kalman estimates)
cov_gap_frac_middle	Fraction of the middle third of the track that is prediction-only	0–1	Trajectory CSV (Kalman estimates)
cov_gap_frac_late	Fraction of the last third of the track that is prediction-only	0–1	Trajectory CSV (Kalman estimates)
cov_gap_clustering_cv	Coefficient of variation of gap row indices	0–∞	Trajectory CSV (Kalman estimates)
cov_high_speed_gap_frac	Fraction of prediction-only rows where speed > 50% of v_max	0–1	Trajectory CSV (Kalman estimates)
cov_mean_speed_at_gaps_mps	Mean estimated speed during prediction-only rows [m/s]	0–40+ m/s	Trajectory CSV (Kalman estimates)
cov_das_hz_est	Estimated DAS update rate [Hz]	0–50 Hz	Kalman measurement audit CSV
cov_cam_hz_est	Estimated Camera update rate [Hz]	0–50 Hz	Kalman measurement audit CSV
cov_gps_hz_est	Estimated GPS update rate [Hz]	0–50 Hz	Kalman measurement audit CSV
cov_das_update_share	Fraction of accepted Kalman updates from DAS	0–1	Kalman measurement audit CSV
cov_cam_update_share	Fraction of accepted Kalman updates from Camera	0–1	Kalman measurement audit CSV
cov_gps_update_share	Fraction of accepted Kalman updates from GPS	0–1	Kalman measurement audit CSV
cov_das_amplitude_mean	Mean DAS raw amplitude A across accepted DAS rows	0–∞ (raw units)	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)
cov_das_amplitude_std	Std dev of DAS amplitude	0–∞	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)
cov_das_amplitude_cv	Coefficient of variation of DAS amplitude (std/mean)	0–∞ (ratio)	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)
cov_das_fiber_dist_mean_m	Mean perpendicular vehicle-to-fiber distance r [m]	0–10+ m	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)
cov_das_fiber_dist_std_m	Std dev of fiber distance [m]	0–5 m	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)
cov_das_fiber_dist_max_m	Maximum fiber distance observed [m]	0–10+ m	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)
cov_das_fiber_dist_cv	Coefficient of variation of fiber distance (std/mean)	0–∞ (ratio)	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)
cov_das_lateral_offset_mean_m	Mean signed vehicle lateral offset from lane centreline [m]	–2 to +2 m	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)
cov_das_lateral_offset_std_m	Std dev of lane-relative lateral offset [m]	0–2 m	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)
cov_das_lateral_offset_max_m	Maximum absolute lateral offset from lane centreline [m]	0–5+ m	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)
cov_das_lateral_offset_abs_mean_m	Mean lateral_offset_m , average unsigned distance from lane centre [m]	0–3 m	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)
cov_das_lateral_offset_range_m	max(lateral_offset_m) – min(lateral_offset_m), total lateral excursion [m]	0–6+ m	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)
cov_das_lateral_frac_outside_half_m	Fraction of DAS rows where lateral_offset_m > 0.5 m	0–1	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)
cov_das_lateral_frac_outside_1m	Fraction of DAS rows where lateral_offset_m > 1.0 m (strong straddling signal)	0–1	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)
cov_das_lateral_sign_changes	Noise-robust sign-change count on smoothed lateral_offset_m, high = lane weaving	0–n	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)
cov_das_lateral_consistent_side_frac	Fraction of DAS rows where lateral_offset_m > 0.3 m and the sign is consistent, high = persistent lane straddling	0–1	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)
cov_das_lateral_change_rate_max	Max Δlateral_offset_m / Δt across consecutive DAS rows [m/s], high = rapid lane-crossing event	0–5+ m/s	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)

A.5 Sensor disagreement (11 features)

Feature	Description (how it is computed)	Output range	Source
dis_das_cam_mean_m	Mean DAS_x – Camera_x when both sensors present [m]	0–20 m	Trajectory CSV (Kalman estimates)
dis_das_cam_max_m	Max DAS_x – Camera_x [m]	0–50+ m	Trajectory CSV (Kalman estimates)
dis_das_hat_mean_m	Mean DAS_x – x_hat [m]	0–20 m	Trajectory CSV (Kalman estimates)
dis_das_hat_max_m	Max DAS_x – x_hat [m]	0–50+ m	Trajectory CSV (Kalman estimates)

dis_cam_hat_mean_m	Mean $ \text{Camera}_x - x_{\text{hat}} $ [m]	0–20 m	Trajectory CSV (Kalman estimates)
dis_cam_hat_max_m	Max $ \text{Camera}_x - x_{\text{hat}} $ [m]	0–50+ m	Trajectory CSV (Kalman estimates)
dis_gps_hat_mean_m	Mean $ \text{GPS}_x - x_{\text{hat}} $ [m]	0–20 m	Trajectory CSV (Kalman estimates)
dis_gps_hat_max_m	Max $ \text{GPS}_x - x_{\text{hat}} $ [m]	0–50+ m	Trajectory CSV (Kalman estimates)
dis_das_only_err_mean_m	Mean pos_err on DAS-only rows [m]	0–20 m	Trajectory CSV (Kalman estimates)
dis_cam_only_err_mean_m	Mean pos_err on camera-only rows [m]	0–20 m	Trajectory CSV (Kalman estimates)
dis_das_worse_than_cam	1 if DAS-only position error exceeds camera-only error	0 or 1	Trajectory CSV (Kalman estimates)

A.6 DAS signal quality (11 features)

Feature	Description (how it is computed)	Output range	Source
das_n_measurements	Total DAS measurements for this track	0–n	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)
das_snr_mean	Mean DAS SNR across accepted DAS rows	0–30+ (ratio)	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)
das_snr_min	Min DAS SNR	0–30+	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)
das_snr_max	Max DAS SNR	0–30+	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)
das_snr_std	Std dev of DAS SNR	0–10+	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)
das_snr_low_frac	Fraction of DAS rows with SNR < 5 (low quality threshold)	0–1	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)
das_sigma_mean_m	Mean DAS position uncertainty σ [m]	0–5 m	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)
das_sigma_max_m	Max DAS position uncertainty σ [m]	0–10+ m	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)
das_confidence_mean	Mean DAS reliability (SNR-derived confidence proxy)	0–1	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)
das_sigma_v_mean_mps	Mean DAS velocity uncertainty σ_v [m/s]	0–5 m/s	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)
das_amplitude_vs_dist_pearson_r	Pearson r between DAS amplitude A and $1/(\text{fiber_dist} + d0)^2$, physics model predicts $A \propto W/(r+d0)^2$, low $ r $ = signal does not follow inverse-square law	-1 to +1	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)

A.7 DAS weight estimation (3 features)

Feature	Description (how it is computed)	Output range	Source
das_W_est_std_kg	Std dev of per-measurement W_{est} series [kg], high = lateral movement or sensor inconsistency	0– ∞ kg	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)
das_W_est_cv	Coefficient of variation of W_{est} (std/mean), normalised instability of weight signal	0– ∞ (ratio)	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)
das_W_est_kg	Estimated vehicle weight [kg] back-computed from mean DAS SNR	0–20 000 kg	Audit CSV, DAS rows (lateral_offset_m, SNR, amplitude)

A.8 Camera and GPS (8 features)

Feature	Description (how it is computed)	Output range	Source
cam_n_measurements	Total camera measurements for this track	0–n	Audit CSV, camera rows
cam_confidence_mean	Mean camera detection confidence	0–1	Audit CSV, camera rows
cam_confidence_min	Min camera detection confidence	0–1	Audit CSV, camera rows
cam_confidence_std	Std dev of camera confidence	0–0.5	Audit CSV, camera rows
cam_low_conf_frac	Fraction of camera rows with confidence < 0.30	0–1	Audit CSV, camera rows
cam_sigma_mean_m	Mean camera position uncertainty σ [m]	0–5 m	Audit CSV, camera rows
gps_n_measurements	Total GPS measurements for this track	0–n	Audit CSV, GPS rows
gps_sigma_mean_m	Mean GPS position uncertainty σ [m]	0–10 m	Audit CSV, GPS rows

A.9 Inter-vehicle interaction (20 features)

Feature	Description (how it is computed)	Output range	Source
iv_n_other_vehicles	Number of other concurrent tracks present during this track's lifespan	0–n	Cross-track Euclidean distance at matched 50 Hz ticks
iv_min_dist_m	Minimum 2-D Euclidean distance to any other track at any shared timestamp [m]	0–200+ m	Cross-track Euclidean distance at matched 50 Hz ticks
iv_mean_min_dist_m	Mean of per-tick minimum distance to nearest other track [m]	0–200+ m	Cross-track Euclidean distance at matched 50 Hz ticks
iv_p10_min_dist_m	10th percentile of per-tick minimum distance, captures	0–100 m	Cross-track Euclidean distance at matched 50 Hz ticks

	sustained close following		ticks
iv_close_proximity_frac	Fraction of shared ticks where nearest vehicle is < 10 m	0–1	Cross-track Euclidean distance at matched 50 Hz ticks
iv_tailgate_proximity_frac	Fraction of shared ticks where nearest vehicle is < 5 m (tailgating threshold)	0–1	Cross-track Euclidean distance at matched 50 Hz ticks
iv_overlap_frac	Fraction of shared ticks where nearest vehicle is within 3 m, sustained = collision	0–1	Cross-track Euclidean distance at matched 50 Hz ticks
iv_overlap_duration_s	Total seconds where nearest vehicle is within 3 m (vehicle-width overlap threshold)	0–∞ s	Cross-track Euclidean distance at matched 50 Hz ticks
iv_ttc_min_s	Minimum Time-to-Collision across the track [s]. TTC = distance / closing_speed, < 2 s is universally the danger threshold	0–∞ s	Cross-track Euclidean distance at matched 50 Hz ticks
iv_ttc_below_2s_frac	Fraction of shared ticks where TTC < 2 s, persistent sub-2-s TTC = chronic tailgating	0–1	Cross-track Euclidean distance at matched 50 Hz ticks
iv_closing_speed_max_mps	Maximum closing speed to any other vehicle at any tick [m/s], kinetic energy proxy for an imminent impact	0–30+ m/s	Cross-track Euclidean distance at matched 50 Hz ticks
iv_time_headway_mean_s	Mean time headway = dist / own_speed [s], < 1.5 s is considered dangerous regardless of absolute distance	0–∞ s	Cross-track Euclidean distance at matched 50 Hz ticks
iv_speed_proximity_ratio_max	Max(own_speed / max(dist_to_nearest, 0.5)), high when fast and very close	0–∞ (ratio)	Cross-track Euclidean distance at matched 50 Hz ticks
iv_speed_excess_over_others_mps	Own mean speed – mean speed of others [m/s], positive = this vehicle is faster than traffic	–∞ to +∞ m/s	Cross-track Euclidean distance at matched 50 Hz ticks
iv_speed_ratio_to_others	Own mean speed / mean speed of concurrent other vehicles, > 1 = faster, < 1 = slower	0–∞ (ratio)	Cross-track Euclidean distance at matched 50 Hz ticks
iv_speed_pearson_r_traffic	Pearson r between own per-tick speed and mean speed of all other vehicles, low = vehicle decoupled from traffic flow	–1 to +1	Cross-track Euclidean distance at matched 50 Hz ticks
iv_decel_at_min_dist_mps2	Longitudinal deceleration of this vehicle at the tick of minimum inter-vehicle distance [m/s ²], collision: high decel coincides with closest approach	–∞ to +∞ m/s ²	Cross-track Euclidean distance at matched 50 Hz ticks
iv_rel_speed_at_min_dist_mps	Magnitude of the relative velocity vector Δv at the tick of minimum inter-vehicle distance [m/s], high relative speed at the closest point = high kinetic-energy collision signature	0–30+ m/s	Cross-track Euclidean distance at matched 50 Hz ticks
iv_collision_risk_proxy	max(Δv ² / max(dist, 0.5)) across all shared ticks, dimensionally ≈ twice the specific kinetic energy of relative motion scaled by proximity, peaks sharply before a collision, stays bounded even at dist → 0 due to 0.5 m floor	0–∞ (m/s ² /m)	Cross-track Euclidean distance at matched 50 Hz ticks
iv_others_mean_speed_when_stopped	Mean speed of all other vehicles during ticks when this vehicle is stopped [m/s], high = anomalous solo stop while traffic flows	0–40+ m/s	Cross-track Euclidean distance at matched 50 Hz ticks